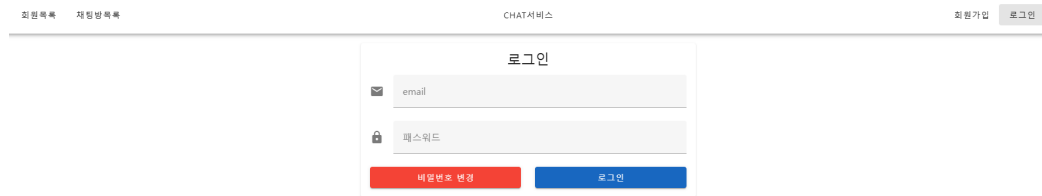


spring, vue 채팅서비스

▼ 수업개요

▼ 주요 서비스

- 회원가입, 로그인



The login form is titled '로그인' (Login). It contains two input fields: 'email' and '패스워드' (Password). Below the password field is a red button labeled '비밀번호 변경' (Change Password) and a blue button labeled '로그인' (Login). The form is part of a web application with a header containing links for '회원목록' (Member List), '채팅방 목록' (Chat Room List), 'CHAT서비스' (CHAT Service), '회원가입' (Sign Up), and '로그인' (Login).

- jwt토큰 기반 로그인 구현

- 채팅서비스

- 실시간 채팅



The chat interface is titled '채팅' (Chat). It shows a conversation between 'hong2@naver.com' and 'hong1@naver.com'. The messages are displayed in a list with timestamps. At the bottom, there is a text input field labeled '메시지 입력' (Message Input) and a blue button labeled '전송' (Send).

- 회원들과 1:1 채팅



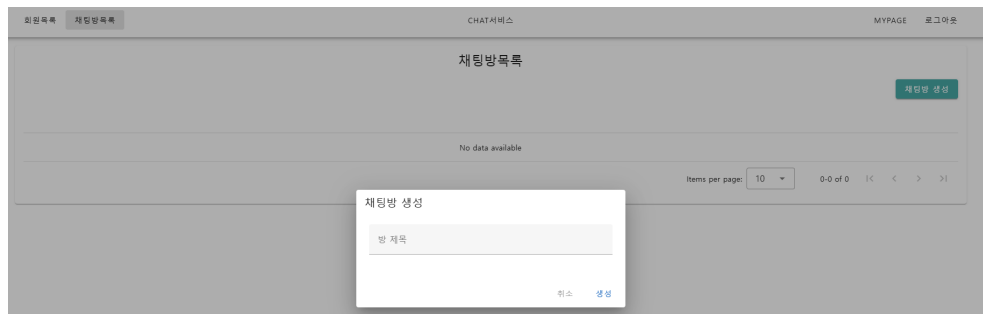
회원목록		
1	admin	admin@test.com
2	hongdong	hong2@naver.com
3	hongdong1	hong1@naver.com
4	hongdong3	hong3@naver.com
5	hongdong4	hong4@naver.com

Items per page: 10 1-5 of 5 < > >>

- 회원목록중에 채팅하고 싶은 사용자에게 1:1 채팅

- 1:1 채팅의 경우 두사람만 채팅할수 있도록 제한

- 단체채팅방



- 누구든 단체채팅방 개설 가능
- 누구든 단체채팅방 list 중에서 원하는 방에 참여 가능
- 내 채팅 목록

CHAT서비스

내 채팅 목록		
채팅방 이름	읽지 않은 메시지	액션
hongildong-hongildong	0	<button>입장</button> <button>나가기</button>
동산동호회	0	<button>입장</button> <button>나가기</button>
독서모임	0	<button>입장</button> <button>나가기</button>

- 내가 참여하고 있는 채팅은 mypage에서 목록으로 조회
- 읽지 않은 메시지 개수 조회 가능
- 채팅방을 나가게 되면 채팅목록에서 제외
 - 모든 사람이 채팅방을 나가게 되면 해당 채팅방은 삭제처리
 - 나가기기능은 그룹채팅일경우에만 활성화

▼ 주요 기술 스택

- java17, spring 3.4, vue3, mysql, redis 기반의 풀스택 개발
- jwt 토큰 기반의 로그인 시스템 구성
- websocket / stomp 기반의 실시간 통신
- redis pub/sub 기반의 다중서버 확장성 설계

▼ 프로젝트 환경구성

▼ docker 설치 및 redis, mysql 실행

- docker desktop 설치 → 모두 기본설정(default)로 설치

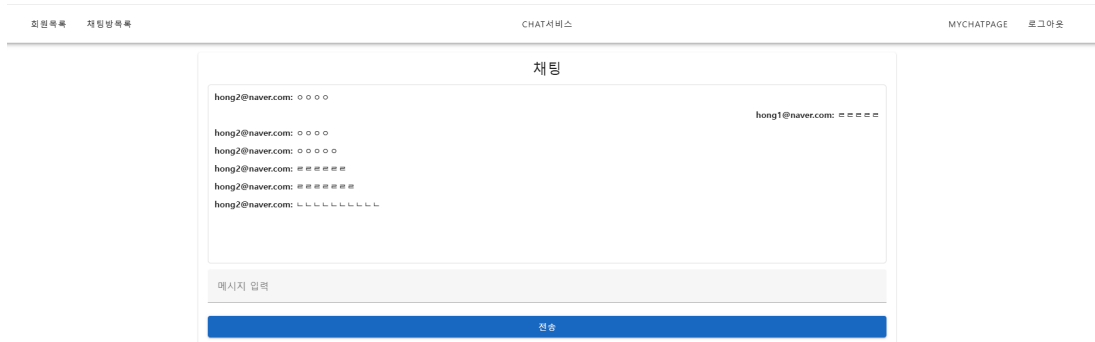
- window 유저의 경우 amd64 설치. mac은 칩에 맞게 설치.
- mysql설치 및 워크벤치 설치
 - 이미 local에 별도로 설치 돼 있다면 해당 db사용 및 db툴 사용해도 무관
 - `docker run --name mysql-container -d -p 3306:3306 -e MYSQL_ROOT_PASSWORD=1234 mysql`
 - 스키마 생성
 - `create database chatserver;`
- redis설치
 - `docker run --name redis-container -d -p 6379:6379 redis`
- ▼ springboot 프로젝트 생성
 - <https://start.spring.io/> 접속
 - 3.4.1 버전, java 17
 - Gradle - Groovy
 - 의존성
 - Spring Web, Spring Data JPA, Spring Security, Lombok, Spring Data Redis (Access+Driver) , MySQL Driver, WebSocket
 - intellij를 통해 build.gradle 파일 open 후 실행
- ▼ vue 프로젝트 생성 및 실행
 - Node.js 설치
 - `npm install -g @vue/cli`
 - -g는 전역설치.
 - `vue create vue-project`
 - 권한 문제 발생시, 파워셸 관리자 모드 실행후 `Set-ExecutionPolicy RemoteSigned`
 - vscode로 프로젝트 open
 - 터미널창에서 `npm install`
 - package.json의 패키지 목록을 설치
 - 의존성 추가

- npm install vue-router, npm install axios, npm install vuexify@3, npm install jwt-decode, npm install sockjs-client, npm install webstomp-client

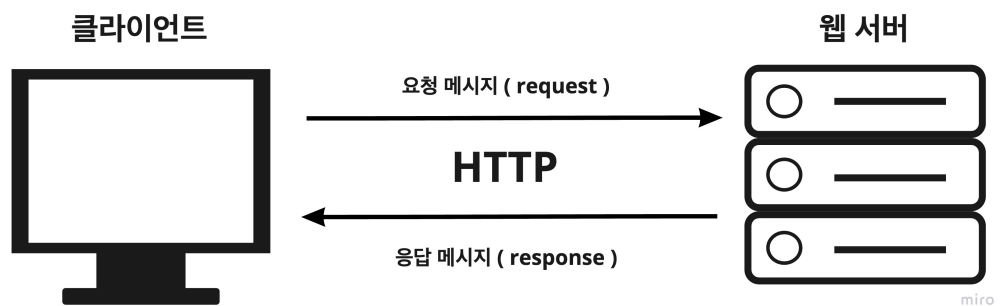
- npm run serve를 통해 실행

▼ 기반기술

▼ websocket

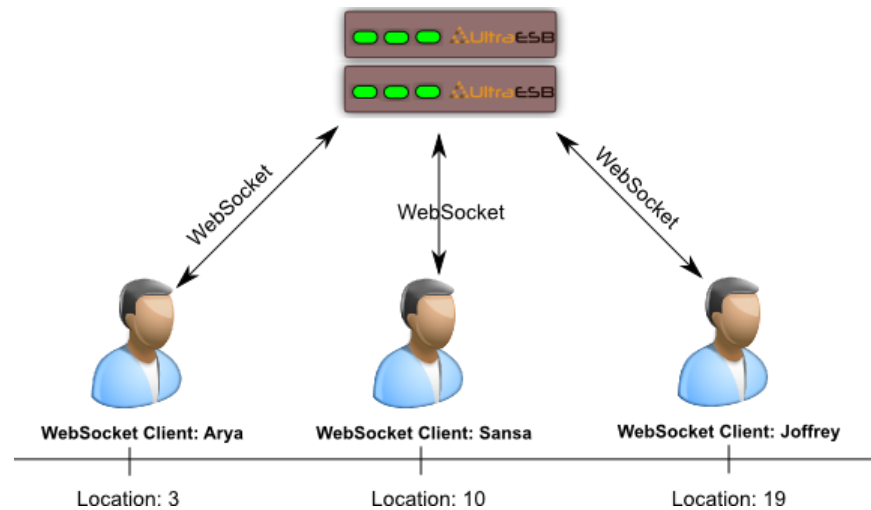


- HTTP 프로토콜의 구조

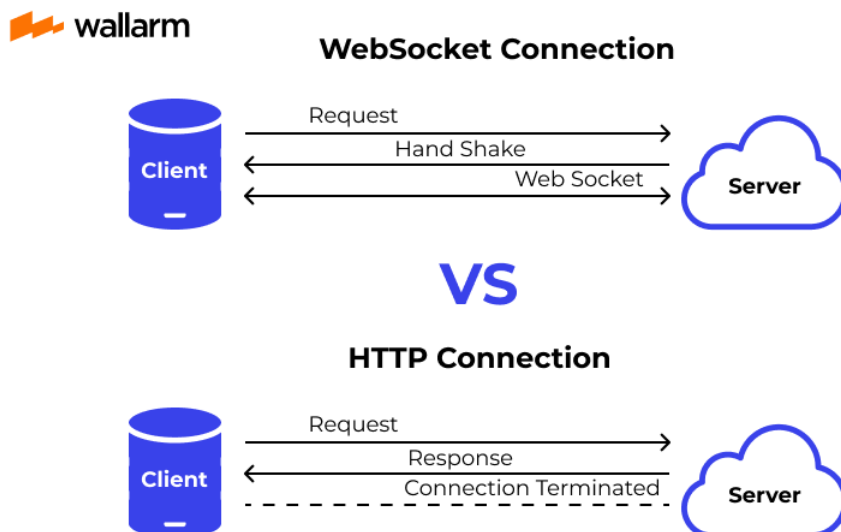


- 클라이언트에서 서버로 요청을 하고 서버가 응답을 주는 단방향 구조

- WebSocket의 통신구조

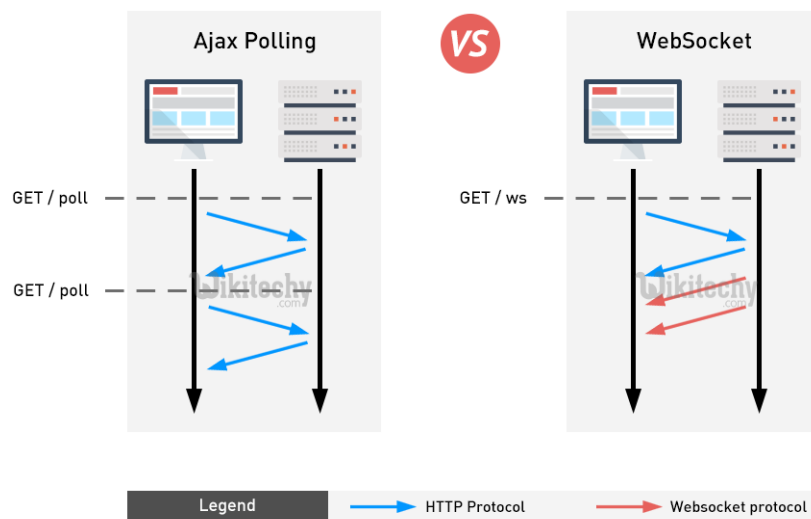


- 클라이언트와 서버 간의 실시간 양방향 통신을 가능하게 하는 통신 프로토콜
- 즉, 클라이언트 → 서버 로의 요청이 아니라, 클라이언트의 요청없이도 서버 → 클라이언트로 정보를 실시간으로 보내줄수 있음을 의미
- HTTP 통신이 아니기 때문에 http:// 대신 ws://의 url을 가진 웹소켓프로토콜을 사용
 - STOMP 메시지 프레임워크를 사용할때에는 명시적으로 ws:// 프로토콜을 사용하지는 않지만, 내부적으로 ws://프로토콜 사용
- HTTP요청과 웹소켓의 서버와의 연결방식 비교

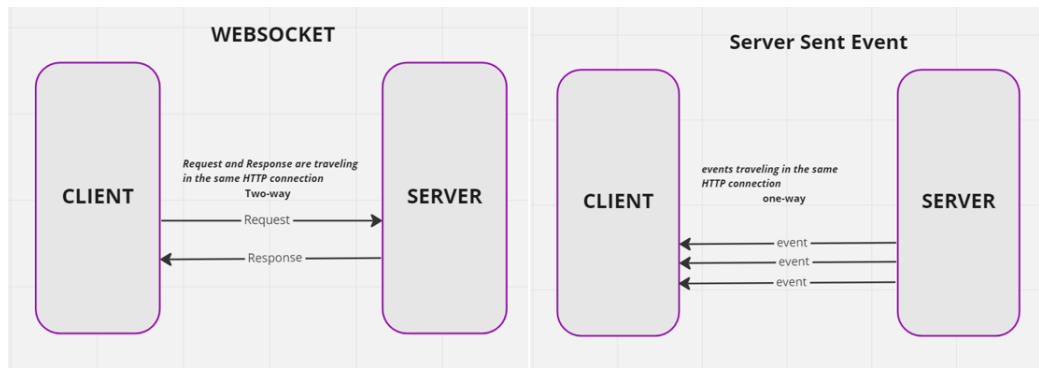


- HTTP는 기본적으로 비연결형 프로토콜로, 클라이언트가 매번 서버에 요청을 보내 새롭게 TCP 연결을 시도하며, 요청 처리 후 연결을 끊음

- 매번 요청을 통해 연결을 맺고 끊어야 하고, 그때마다 http메시지를 보내야 하므로 지속적 통신에 있어서 성능저하 발생
- 웹소켓은 연결형 프로토콜로, 최초 1번만 연결이 성공적으로 이루어지면 WebSocket 프로토콜로 업그레이드하여 TCP 연결 유지
 - 한번만 연결을 맺고 나면 그 뒤엔 별도의 http메시지 없이 통신이 가능해서 빠른 성능 보장
 - 웹소켓도 최초 한번의 연결시 HTTP 헤더를 통해 token과 같은 인증 정보를 보낼수 있고, 이때에 이를 통해 인증처리를 할수 있음
- Polling방식과 websocket 방식의 비교



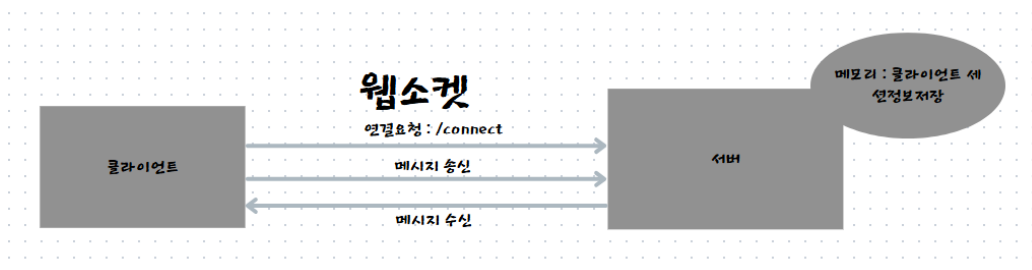
- Polling
 - 클라이언트가 서버에 주기적으로 요청을 보내 업데이트를 확인하는 방식
 - HTTP 요청만 처리하면 되므로 추가 설정이 필요없는 간결함
 - 다만, 지속적인 서버 request로 서버 과부하 가능성 존재
- WebSocket
 - 클라이언트와 서버 간 지속적인 연결을 유지
 - 무거운 HTTP 메시지가 필요 없이 직접 메시지를 주고 받음으로서 서버 부하 측면에서 우수
- SSE기술과 Websocket 비교



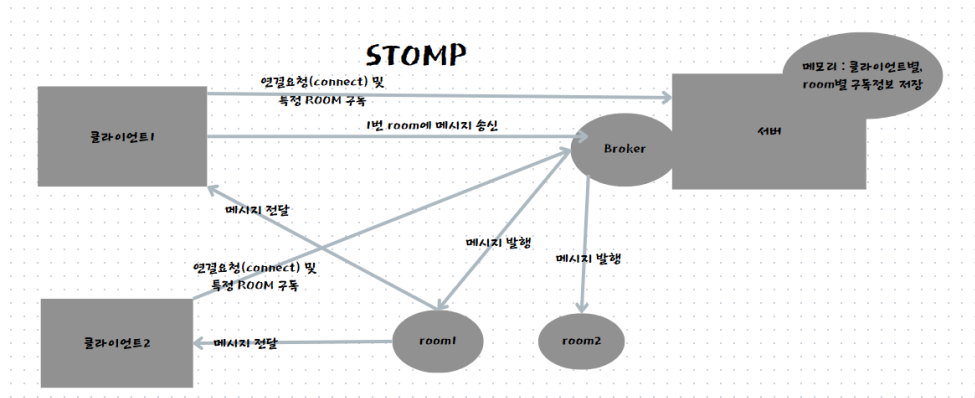
- SSE(Server-Sent Events) 기능은 HTTP기반의 클라이언트와 서버 간의 실시간 단방향 통신을 가능하게 하는 웹 기술
- 웹소켓과의 차이
 - sse프로토콜을 사용한 통신방향이 서버 → 프론트엔드로의 단방향 통신
 - 프론트에서 서버로의 데이터 전송은 일반적인 HTTP통신
- 서버는 클라이언트에게 지속적으로 데이터를 전송할 수 있으며, 주로 실시간 업데이트가 필요한 애플리케이션에서 사용(특히, 사용자에게 알림을 주기 위한 목적으로 많이 활용)

▼ stomp

- STOMP(Simple Text Oriented Messaging Protocol)는 WebSocket 위에서 동작하는 메시징 프로토콜
 - 일반 웹소켓

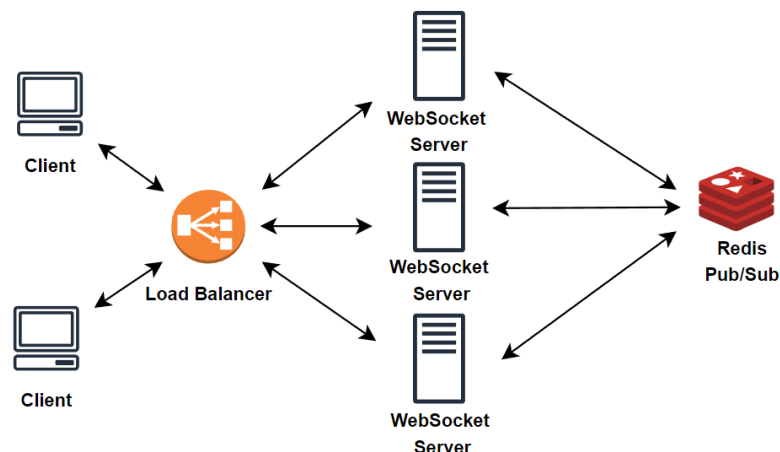


- STOMP



- Stomp는 websocket과 다르게, 목적지기반 메시지 라우팅 지원
 - 클라이언트와 서버가 특정 주제(topic) 또는 경로를 기준으로 메시지를 교환하는 구조
- 메시지 교환 절차
 - 클라이언트에서 지정된 특정 roomId에 메시지를 발행하면 broker에 의해서 해당 roomId 채널에 메시지가 전달
 - 동시에 roomId를 구독하고 있는 클라이언트에게 실시간으로 메시지가 전달

▼ redis pub/sub을 통해 멀티서버 환경 고려

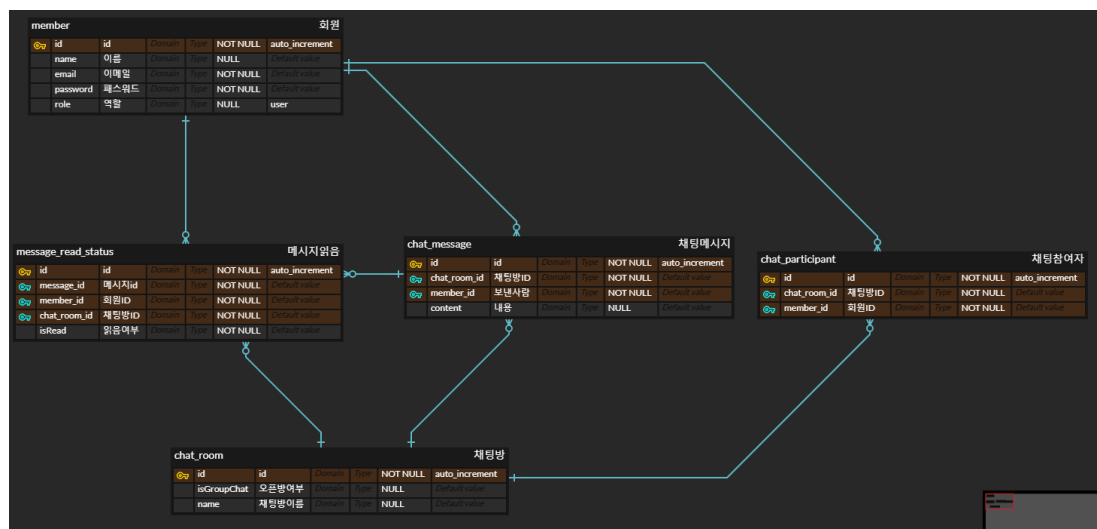


- 웹소켓 통신에서 특정 사용자는 특정 서버의 메모리에 의존적이어서, 클라이언트A가 발행한 메시지를 클라이언트B는 다른 서버에 연결되어 받지 못할수 있음
- 이러한 멀티 서버 환경에서 원활한 채팅을 위해서 메시지를 서버에 모두 전파해주는 redis pub/sub기능 사용
- 메시지 발행, 구독 절차

- 클라이언트는 송신할 메시지를 websocket server에 송신
- websocket server는 해당 메시지를 곧바로 특정 topic에 메시지를 발행하지 않고, redis의 pub/sub 기능을 활용하여 메시지를 모든 서버에 publish
- 모든 서버는 redis를 subscribe하고 있기에 redis로부터 발행된 message를 받아서, 본인 서버의 topic에 메시지를 발행
- 각 서버를 subscribe하고 있는 클라이언트들은 특정 room에 전파된 message를 수신

▼ 개발 및 테스트 방법

▼ 채팅서비스를 위한 ERD설계

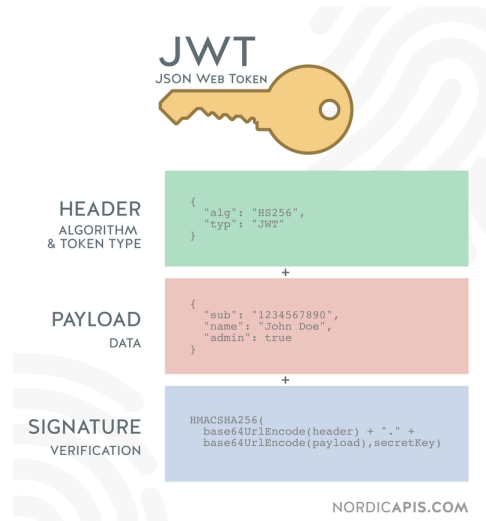


- ERD 클라우드에서 실습 진행
- 회원, 채팅방, 채팅메시지, 채팅참여자, 메시지읽음 총 5개의 테이블로 구성

▼ 백엔드

▼ 회원파트

- 회원가입, 회원목록조회 api개발
 - 회원가입시 비밀번호 암호화를 위한 passwordencoder생성
- jwt 기반 로그인



- security configs 설정을 통해 cors, 로그인 예외 설정 등 진행
- login api를 통해 로그인 성공시 jwt 토큰 발급
- jwtfilter를 생성하여 추후 api요청시 토큰 검증

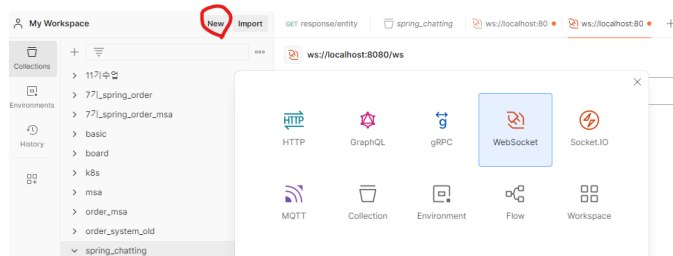
▼ 채팅파트

▼ 순수 웹소켓을 통한 통신 실습

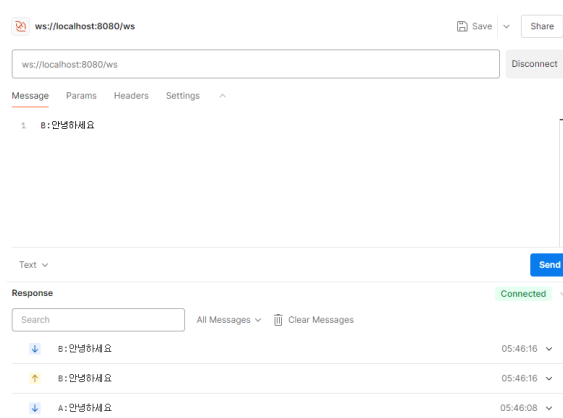
- 'org.springframework.boot:spring-boot-starter-websocket' 의존성
 - 웹소켓, stomp 등을 포함하고 있는 라이브러리
- WebSocketConfigurer클래스를 구현한 WebSocketConfig 클래스 생성
 - WebSocketConfigurer는 STOMP와 같은 메시지 브로커 없이 기본 WebSocket 연결을 설정하는 데 사용
 - WebSocketConfigurer는 WebSocket 엔드포인트(/connect)를 설정
- TextWebSocketHandler 클래스를 구현한 SimpleWebSocketHandler 클래스 생성
 - connect로 연결요청이 들어왔을때 해당 요청을 처리할 클래스
 - 연결전, 연결시, 연결후 처리를 손쉽게 진행
- 채팅 코드 테스트
 - postman등을 통해 ws://localhost:8080/ws 엔드포인트로 채팅 실습 가능

■ ws프로토콜 테스트

- websocket 선택 후 url엔드포인트로 연결



- 메시지 송수신



- 메시지를 받을때는 아래 화살표, 메시지를 보낼때는 위 화살표

- Postman은 순수 websocket은 지원하나, STOMP 테스트는 지원하지 않음에 유의

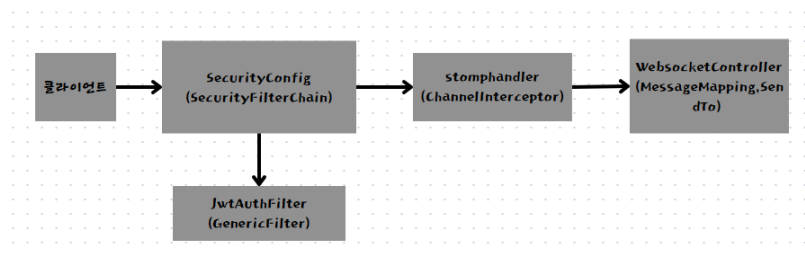
■ stomp 테스트

- postman에서는 stomp는 지원하지 않고, 과거 지원하던 apic 크롬 확장 프로그램도 현재는 비활성화
- 테스트의 제약으로 stomp테스트는 백엔드 개발자라 할지라도 간단한 프론트 엔드 화면 구성후 테스트하는 것을 권고

▼ stomp를 통한 통신 실습

- WebSocketMessageBrokerConfigurer를 구현한 StompWebsocketConfig 클래스 생성

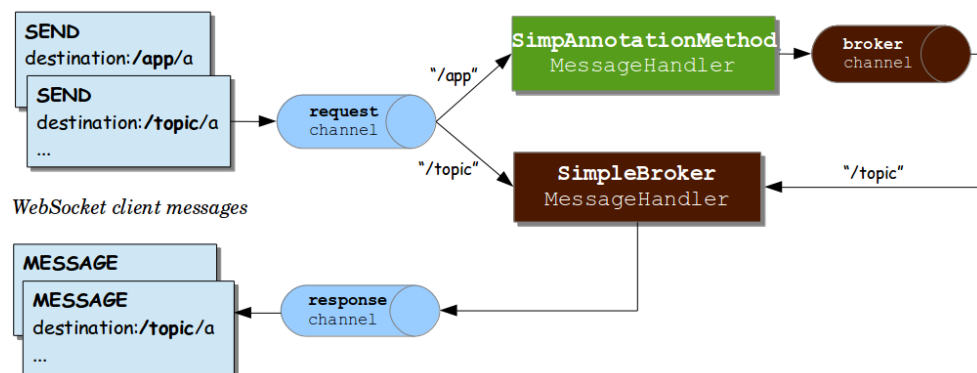
- WebSocketMessageBrokerConfigurer는 STOMP 프로토콜을 기반으로 WebSocket 메시지 브로커를 설정하고 구성할 때 사용
- 주요 기능
 - 최초 연결을 맺기 위한 endpoint를 정의
 - MessageBrokerRegistry를 통해 메시지를 발행, 구독하기 위한 url패턴(prefix)를 정의. 이 클래스는 simplebroker라고 불리고, 메시지를 특정 topic에 연결해주는 브로커 역할 수행.
 - interceptor가 필요하다면 interceptor클래스를 추가
- ChannelInterceptor를 구현한 StompHandler 클래스 생성
 - ChannelInterceptor는 클라이언트가 서버로 전송하는 STOMP 메시지(CONNECT, SUBSCRIBE, SEND, DISCONNECT 등)를 가로채는 인터셉터
- StompController클래스 생성
 - 실습1
 - MessageMapping, SendTo 어노테이션 활용한 한 메서드 내에서 간단한 메시지 라우팅
 - 실습2
 - MessageMapping과 SimpMessageSendingOperations클래스의 convertAndSend기능을 활용하여 송신 메서드, 수신 메서드 분리



- EventListener클래스 생성
 - 스프링과 STOMP는 기본적으로 세션 관리를 자동으로 처리

- 클라이언트가 /connect 엔드포인트를 통해 연결될 때 세션이 생성
 - 클라이언트가 연결을 종료하거나 타임아웃이 발생하면 세션이 삭제
 - 세션관련하여 추가적인 작업이 필요한 경우 EventLisnter 생성
 - 연결/해제 이벤트를 기록
 - 연결된 세션 수를 실시간으로 조회
- 채팅 코드 테스트
 - stomp기반의 통신은 현재 postman에서는 지원하지 않고, 이를 지원하는 외부 서비스가 마땅치 않으므로, 간단한 프론트엔드 구현과 함께 테스트를 하는 것을 권장

▼ spring에서의 stomp 주요 요소



- 위 그림의 메시지 교환 절차
 - 클라이언트에서 지정된 /app(publish)/{roomId} 경로로 메시지를 발행하면 broker에 의해서 /topic/{roomId} 이라는 경로의 채널에 메시지가 전달
 - 동시에 /topic/{roomId}를 구독하고 있는 클라이언트에게 실시간으로 메시지가 전달
- SimpAnnotationMethodMessageHandler
 - 우리 코드상의 StompController에서 @MessageMapping과 @SendTo와 같은 애노테이션이 선언된 메서드를 처리하는 핸들러
 - 어노테이션을 통해 지정된 메서드로 메시지를 라우팅하는 역할

- SimpleBroker(브로커)
 - Spring에서 제공하는 메모리 기반 브로커로서, 경로에 따라 메시지를 분배하고 클라이언트에게 메시지를 전달
 - SimpleBroker (내장 브로커-enableSimpleBroker), StompBrokerRelay (외부 메시지 브로커-enableStompBrokerRelay) 사용
 - 우리 코드 상에 StompWebSocketConfig에서 configureMessageBroker를 통해 설정

▼ 채팅 API개발(메시지 저장/조회, 채팅방관련 추가/조회 등)

- ChatRoom, ChatParticipant, ChatMessage, MessageRead 도메인 생성
- 각 도메인에 맞는 repository 생성
- api 목록(ChatController)
 - 메시지저장(saveMessage 메서드구현)
 - 그룹채팅관련
 - 그룹채팅방 개설(/chat/room/group/create)
 - 그룹채팅 목록조회(/chat/room/group/list)
 - 그룹채팅에 참여자추가(/chat/room/group/{roomId}/join)
 - 1:1채팅관련
 - 개인채팅방 개설(/chat/room/private/create)
 - 공통처리
 - 이전메시지 내역조회(/chat/history/{roomId})
 - 채팅방메시지 읽음처리(/chat/room/{roomId}/read)
 - 내 채팅방 목록조회(/chat/my/rooms)
 - 채팅방 나가기(/chat/room/group/{roomId}/leave)

▼ 프론트엔드

▼ 회원파트

- 홈, header화면 추가(HeaderComponent.vue)
- 회원가입화면 추가(MemberCreate.vue)

- 로그인화면 추가(LoginPage.vue)
- 회원목록조회화면 추가(MemberList.vue)

▼ 채팅파트

- simple websocket 화면 추가(SimpleWebSocket.vue)
- stomp websocket 화면 추가(ChatPage.vue)

▼ 채팅 API 관련

- groupchat 목록 화면(GroupChattingList.vue)
 - 그룹채팅 목록조회
 - 그룹채팅방 개설 버튼
 - 그룹채팅 참여시 그룹채팅에 참여자추가 후 채팅방 입장
- 채팅화면 수정
 - 이전메시지 내역 불러오기
 - 채팅방 나갈때에 채팅방메시지 읽음처리
- 회원목록조회 화면에 채팅 버튼 추가
 - 1:1 채팅이므로, 개인채팅방 개설 후 채팅방으로 입장
- 내 채팅방 목록조회 화면 추가(MyChatPage.vue)
 - 채팅방입장 및 나가기 기능 버튼 추가

▼ redis pub/sub 구현

- redis config 생성
 - redis 연결객체인 Lettuce기반의 ConnectionFactory 객체 생성
 - 일반적인 String 값을 key, value 메시지 발행을 위한 StringRedisTemplate 객체 생성
 - RedisMessageListenerContainer객체를 생성하여 Redis의 특정 채널을 구독
 - MessageListenerAdapter를 사용하여 수신된 메시지 처리 (onMessage)
- 메시지 송수신 절차
 - StompController를 통해 send된 message를 redis로 publish

- redis의 "chat"채널로 메시지가 발행되면, 이 채널을 구독 중인 RedisMessageListenerContainer가 메시지를 수신
- 수신된 메시지는 MessageListenerAdapter를 통해 onMessage 메서드로 전달
- onMessage 메서드는 수신된 메시지를 각 서버의 topic에 메시지를 발행하여, 이를 subscribe하고 있는 클라이언트에게 채팅 메시지 전달